Can reinforcement learning beat the best scheduling rule?

We trained neural networks to schedule cloud jobs. They learned to copy good rules of thumb, but on fresh test instances none was significantly better than the best rule.

Ty Riekert | Curtin University, Year 3 research project | Supervisors: Elham Mardaneh, Tony Mathew

WHY IT MATTERS

Deciding what to run first, and where, shows up everywhere

The problem

Work arrives over time, each piece with a deadline and a cost if it is late

Something must decide which piece goes next and on which machine

Good choices mean fewer late jobs and less energy spent keeping machines on

Same problem, other fields

Data centres

which task runs where, and when machines can be switched off

Hospitals

which operation or scan comes next, in which room

Factories and logistics

which order is built or delivered first

Cloud cost and energy

a business can trade lateness against running cost

Our test case: cloud computing jobs on a group of machines. The same structure appears in the fields on the right (by analogy; we tested only the cloud case).

WHAT WE DID

We compared classic rules with a neural network that learns to schedule

1

Classic rules

Simple, fast recipes such as "do the most urgent job first". Used in practice today.

2

A learned scheduler

A neural network trained by trial and error (reinforcement learning), or first taught by copying a rule, then improved.

3

A fair test

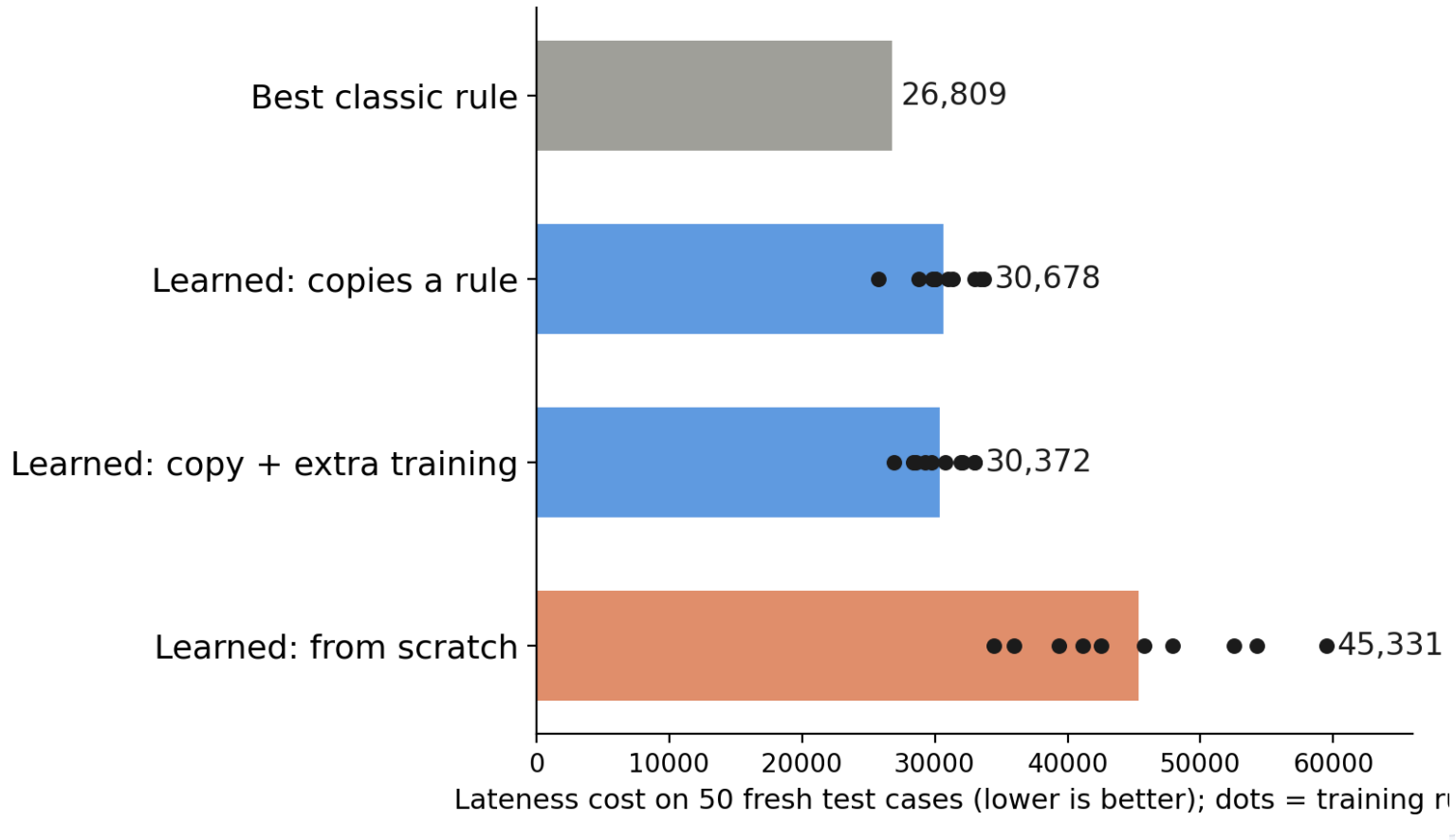
Both tried on 50 fresh test cases that no design decision had ever seen, scored on lateness, late jobs and energy.

The question: can the learned scheduler beat the best classic rule?

Rules named in the report: EDF (earliest deadline first), MDC, COVERT, WLST, ATC. PPO is the reinforcement learning method used.

WHAT WE FOUND

No learned scheduler was significantly better than the best classic rule



0 of 3

learned setups significantly better than the best rule

Close

copies of a rule finish near it; extra training did not help

Wide

learning from scratch varied a lot between runs

Online, 10 machines, load 0.95; 50 fresh test cases; exploratory results; 10 training runs per learned setting.

TRY IT

Which scheduler is best for what you care about?

1

Choose the environment

How busy the machines are, how tight the deadlines are, online or offline

2

Choose what matters

Energy, lateness, or your own weighted mix of metrics

3

Press Start

The two best rules race the two best learned models; 100% = the best score of that scenario

Things to notice

The winner changes with the goal and with the load. Learned models are mostly copies of a rule, so they finish beside it. What would you weight?

WHAT I LEARNED

Skills built this year, honest limits, and what comes next

Skills developed

Reinforcement learning and imitation (PPO, behaviour cloning)

Experiment design: fresh test sets, paired comparisons, pre-set wording rules

Research software: queued training runs, reproducible results archive

Communicating negative results honestly

Honest limits

Few training runs per setting (3-10)

Best rule was picked on the original test set

Extra training tried at one tuned setting only

Exploratory, not proof that RL cannot win

Next

One objective that weights all metrics together

Explain why from-scratch learning is unstable

[Add your own career goal here]

Details and per-seed values: Future/research/2026-10-21-final-week-findings.md